

هنبداً المشروع خلاص المرة اللي فاتت كنا بس بنفهم ايه الخطوات بس مبنشتغلش كده

N-Tier Architecture

عاوز اعمل طريقة انى ابني بيها المشروع
ازاى هنحط كل جزء في البروجكت؟ وهعمله ازاى؟

اللي عملناه المرة اللي فاتت ده غلط لان كله في نفس البروجكت وده غلط المفروض كل جزء في بروجكت لوحده
جزء يتعامل مع الداتا بيز - جزء مع الريكوستس - جزء مع البيزنيس
(ممكن يكون عندى اجزاء اكثر بس ده ابسط الاجزاء)

ايه المشاكل في الطريقة اللي فاتت؟
ال coupling لانى يعتمد علي ال dbContext الخاص بالسيرفر معين
لازم شغلك يعتمد علي الانترفيس
الكود مش هعرف اعمله تيستينج لان تيم التيستج بيشتغل علي داتا فيك
كل شوية ه تلاقى نفسك بتكرر ال LINQ queries
مش فاصل بين الاجزاء

The Hidden Problems

Tight Coupling

Controller **knows** EF Core, the `DbContext`, the connection string — everything is welded together.

Untestable

Want to test the controller without hitting a real database? **You can't**. The `new` hardcodes the dependency.

Code Duplication

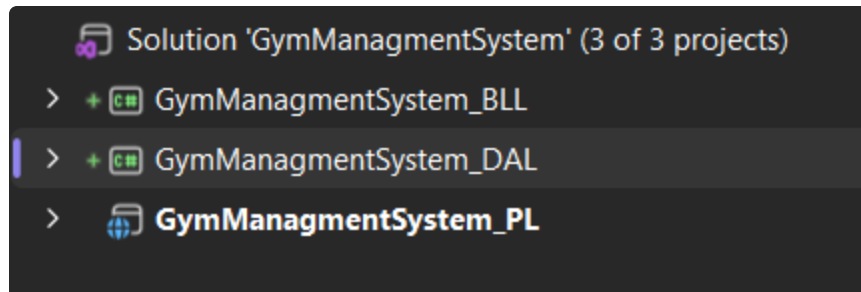
Every controller writes the **same LINQ queries** — there's no shared place for "get all active records" logic.

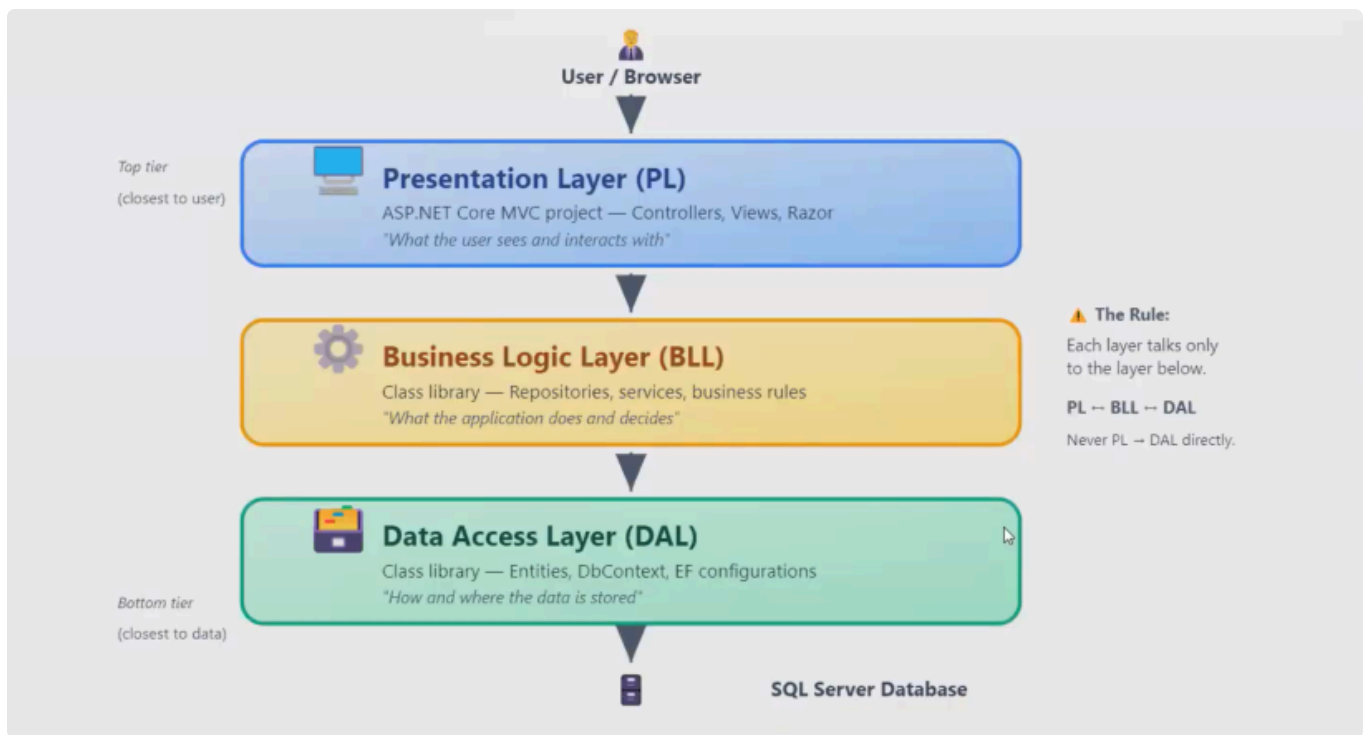
No Separation

UI logic, business rules, and DB queries all live in **the same project**. Hard to reuse, hard to swap parts.

فكرة ال Arch اي كان نوعها انها تحل المشاكل وتقسم المشروع بطريقة معينة

احنا هنستخدم ده عشان هو ابسط واحد مستخدم مع المشاريع الصغيرة لان فيه مشاكل وهنفهمها
مش ده النوع الوحيد عندنا انواع كتيرة
خلي بالك ان ال 2 layers التانيين هم class lib

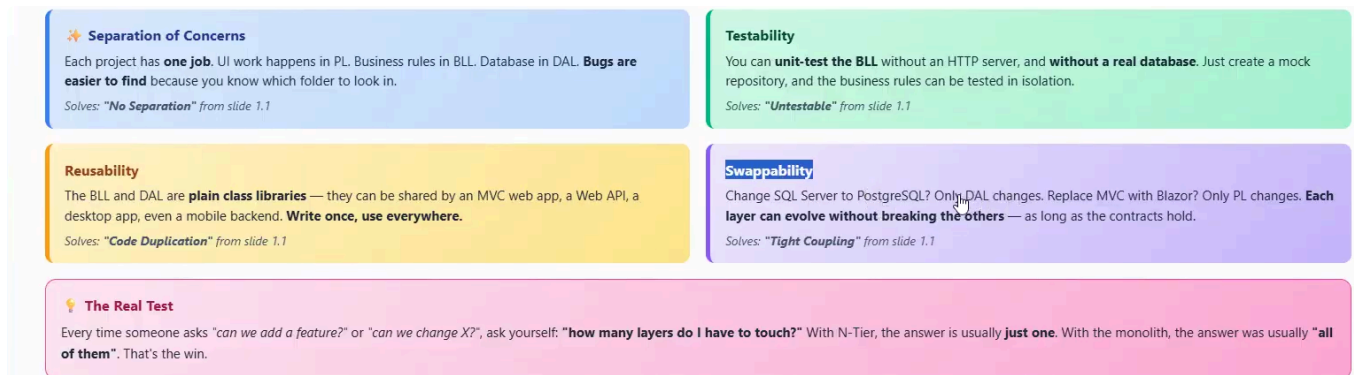




مبحثش في نفس ال layer بكون فيه implementation, interface وهو ده سبب انه وحش وعنده مشاكل وهو بيعمل ده مع ال REPOSITRIES



المميزات



Repository Pattern

مين ده وعمل ايه؟

قبله كنت بكرر Linq operators كتيرة + كنت بتعامل مع ال db context direct محتاجة افصل بينهم عشان ميعرفش يعمل اكسيس ويكون معاه full features

فمثلا كان عندنا ال plan Controller كان بيكلم ال dbContext direct

```
public class PlansController : Controller
{
    3 references
    private readonly GymDbContext gymDbContext;
    0 references
    public PlansController()
    {
        gymDbContext = new GymDbContext();
    }

    0 references
    public async Task<IActionResult> Index()
    {
        var plans = await gymDbContext.Plans.ToListAsync();

        return View(plans);
    }
}
```

فده غلط

عشان اصلح ده هكلم LAYER هي اللي تكلمه فهكلم ال REPOSATRY

المفروض تكون موجودة في ال DATA ACCSESS LAYER

فهنبندی نصلح الدنيا

منساش ان ال sql server و ال tool مش هحتاجهم في ال pl احتياجي ليهم في ال DAL

فكده ال DAL هيكون فيه :

CONTEXT - MODELS - Configurations-Reposatrics

فالوقتني انا عاوز اعمل ريبوزاتري عشان مكلمش ال dbContext direct

جواها حاجتين classes - interfaces

كل مودل عندي هيكلم الداتا بيز دايركتلي فهيكون له الانترفيس الخاص بيه

عاوزه احط فيه ال signature بتاعة ال 5 crud operations main

البروجكت شغال ASYNC فكلهم بيرجعوا Task

```

namespace GymManagmentSystem_DAL.Repositories.Interfaces
{
    0 references
    public interface IPlanReposateries
    {
        0 references
        public Task<IEnumerable<Plan>> GetAllPlansAsync(bool tracking = false);

        0 references
        public Task<Plan?> GetByIdAsync(int Id);

        0 references
        public Task<int> AddAsync(Plan plan);

        0 references
        public Task<int> UpdateAsync(Plan plan);
        0 references
        public Task<int> DeleteAsync(Plan plan);
    }
}

```

ده كده الانترفيس هروح اعمل ال class عشان اعمل implement methods هلاقي انهم محتاجين ال dbcontext عشان اكلم الداتا بيز

```

1 reference
public async Task<int> AddAsync(Plan plan)
{
    await _dbContext.Plans.AddAsync(plan);
    return await _dbContext.SaveChangesAsync();
}

```

وبكده ال controller محتاج اني اخد ريفرنس من ال plan repository

```

1 reference
private PlanReposatry PlanReposatry;

0 references
public PlansController()
{
    PlanReposatry = new PlanReposatry();
}

```

بس كده انت لسه عندك مشكلة مش بتطبق ال dependency لانك عامل بناء علي ال class احنا بنحب يكون بناء علي ال interface

```

3 references
private IPlanReposateries PlanReposatry;

0 references
public PlansController()
{
    PlanReposatry = new PlanReposatry();
}

```

خلي بالك ان ال pl بيكلم ال bli بس هنغيرها بعدين

Dependency Injection

--خلي بالك فيه عك اتعمل

انى انا اللي اعمل create object لان في الريبكوست ممكن اكلم اكتر من ريبو وكل ريبو محتاج يكلم ال dbcon فانت كده فاتح اكتر من سيشن وده مش صح

انا عاوز افتح سيشن واحدة بس

فبدل كل مرة ما اعمل create object هعمل واحد بس والكل يستخدم نفس الاوبجكت وعشان ده يحصل مش انت اللي هتعمل الاوبجكت مشكلتي هنا ان انا اللي عاملة الاوبجكت

```

1 reference
public PlanRepository()
{
    _dbContext = new GymDbContext();
}

```

مين مسؤل انه هو اللي هيعمله هو ال clr هو اللي هيعمل الاوبجكت و ده اسمه ال dependency

```

1 reference
public PlanRepository(GymDbContext dbContext)
{
    _dbContext = dbContext;
}
1 reference

```

بس لسه مقزلتش لل clr ازاي يعمله؟

lifetime in heap

 AddScoped "per HTTP request" Lifetime: One instance per request. Same request → reuses the instance. Next request → fresh instance. ✓ Use for: - Repositories - DbContext (default) - Services that hold per-request state	 AddSingleton "once for the entire app" Lifetime: One instance forever. All requests share the same instance. Created on first use, lives until the app stops. ✓ Use for: - Stateless utilities - Caches, loggers - Configuration readers ⚠ Avoid: per-user state (data leaks!)	 AddTransient "new instance every time" Lifetime: Fresh instance every injection. Even within the same request, two injections = two different objects. ✓ Use for: - Lightweight, stateless helpers - Mappers, validators - Anything cheap to construct ⚠ Avoid: heavy classes (overhead!)
--	--	--

هختار addscoped افضل نوع عشان مبيسبش الكونيكشن مفتوح

فكده هقول لل clr يعمل كريت لاوبجيكيت منه

المكان الصح لل connection string مش ال dbcontext عشان لو عاوزة انقل بينهم مش هيساعدني مكانه الصح في ال appsettings

```
PlansConfigurations.cs  PlanReposatry.cs*  GymDbContext.cs*  PlansController.cs*  Program.cs*  appsettings.json*
Schema: https://www.schemastore.org/appsettings.json
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10    "DefaultConnection": "Server=. ;Database= Gymdbv2; Trusted_Connection = True; TrustServerCertificate= True;"
11  }
12 }
13 }
```

واعدل ال dbcontext عشان افتح connection مع الداتا بيز

```
public class GymDbContext : DbContext
{
    0 references
    protected GymDbContext(DbContextOptions<GymDbContext> Options):base(Options)
    {
    }
    0 references
```

ممکن ابعت connection string مع ال usesqlserver

```
options.UseSqlServer("Server = .; Database=GymManagement; Trusted_Connection = true ; TrustServerCertific

بس برضو ده مش افضل عشان مش هتعرف تغير الكونفيجریشنز بسهولة فالصح اني اقراه من ال appsettings

builder.Services.AddDbContext<GymDbContext>(options =>
{
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
});
```

برضو محتاجين ن allow dependency للريبو

```
builder.Services.AddScoped<IPlanReposateries, PlanReposatry>();
```

سؤال انترفيو مهم

Dependency inversion vs Dependency Injection

Dependency inversion: وهو ان الكلاس او السيرفيس بيعتمد علي الانترفيس مش الكلاس solid مبدأ في ال

Dependency Injection: هو اللي هيعمل كريت لاوبجيكيت لل clr ان ال

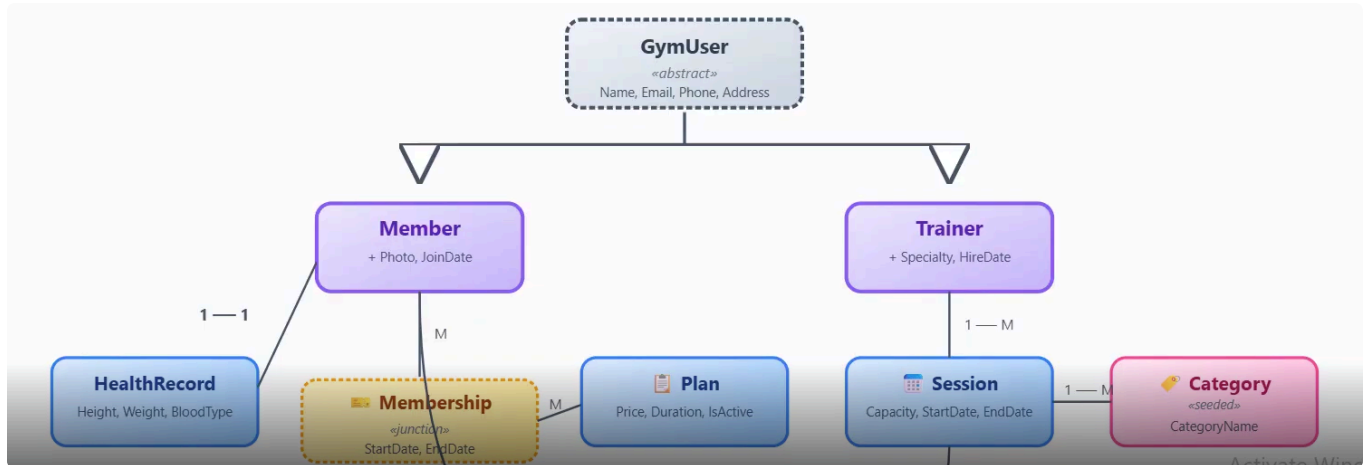
مش انا اللي هعمل كريت لاوبجيكيت يقلل ال coupling , يقلل الاعتمادية, الاوبجيكيت يعيش اكثر من فترة

Models

عاوزين نبني الداتا بيز بتاعتنا

بعد ما اخلص الاركتيكتشر اروح اعمل ال DAL

انا حاليا معنديش غير كلاس واحد بس وهو ال plan



المودلز مشتركين في ال id وشوية حجات تانية

فهروح اعمل كلاس ال base entity الغرض منه احط كل الكولومز المشتركة فيه

وكمان فيه غرض ثاني مهم جدا جدا

```
namespace GymManagementSystem_DAL.Models
{
    1 reference
    public abstract class BaseEntity
    {
        0 references
        public int Id { get; set; }
        0 references
        public DateTime CreatedAt { get; set; }
        0 references
        public DateTime? UpdatedAt { get; set; }
    }
}
```

ال member , trainer مشتركين فيه :

- **Id** → Unique identifier for the trainer.
- **Name** → Full name of the trainer.
- **Email** → Contact email address.
- **Phone** → Contact phone number.
- **DateOfBirth** → Trainer's date of birth.
- **Gender** → Gender of the trainer.
- **Address** → Structured address containing:
 - Building No
 - Street
 - City

فيمكن اعمل حاجة كده اسمها ال gymUser
خلي بالك عشان ميحولش الكلاس بتاع ال address ل table
محتاجة اقوله ان هو [owned]

```
0 references
public abstract class GymUser : BaseEntity
{
    0 references
    public string Name { set; get; } = default;
    0 references
    public string Email { set; get; } = default;
    0 references
    public string Phone { set; get; } = default;

    0 references
    public DateOnly DateOfBirth { set; get; };

    0 references
    public Gender gender { set; get; };

    0 references
    public Address address { set; get; } = default;
}
[Owned]
1 reference
public class Address
{
    0 references
    public string street { set; get; } = default;
    0 references
    public string city { set; get; } = default;
    0 references
    public int BuildingNumber { set; get; } = default;
}
}
```

Configurations

مبدائيا فيه configurations مشتركة بين الداتا
هحط كونسرتين عشان اخليه للميمبر والتريزر بس

```
0 references
public class GymUserConfigurations<T> : IEntityTypeConfiguration<T> where T : GymUser: new()
```

بعد ما بخلص كل الكونفيجريشنز بكون عاوزة اعمال apply ليهم كلهم فيروح لل dbContext
ده هيعمل كله الكونفيجريشنز

```
0 references
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.ApplyConfigurationsFromAssembly(Assembly.GetExecutingAssembly());
}

6 references
public DbSet<Plan> Plans { get; set; }
0 references
public DbSet<Trainer> Trainers { get; set; }
0 references
public DbSet<Member> Members { get; set; }
0 references
public DbSet<Category> Categories { get; set; }
0 references
public DbSet<Session> Sessions { get; set; }
}
```